ST-1
ST-2
ST-3

Tree Flattening

Q. Given an Array. Q queries

Each query

1. i, val ⇒ a[i] = val
2. l, r ⇒ Find minimum in the range [l, r]

	0	1	2	3	4	5	6	7
arr[] =	10,	2,	~~7~~ -7,	-3,	5,	8,	1,	15

Q = 4

1. II L = 1, R = 4 ⇒ -3
2. II L = 4, R = 6 ⇒ 1
3. I j = 2, val = -7
4. II L = 0, R = 5 ⇒ -7

BF:

1. make a[i] = val TC: O(1)
2. L, R
   Iterate & find minimum. TC: O(N)

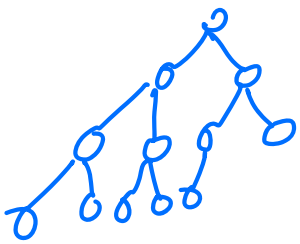
TC: O(Q.N)

N → no of elements in array

Q → Queries.

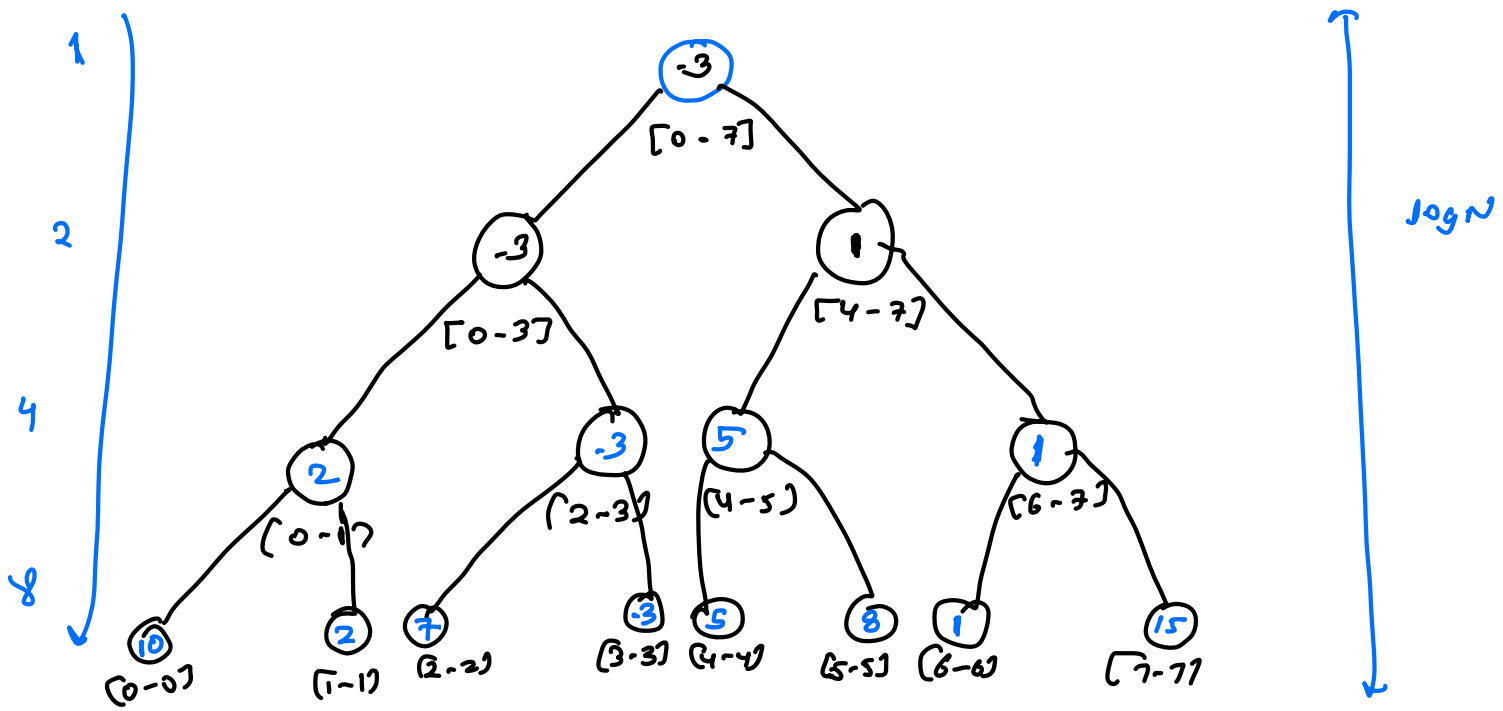
Segment Tree

1. Complete Binary Tree



0 1 2 3 4 5 6 7

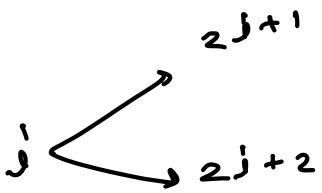
arr = [10, 2, 7, -3, 5, 8, 1, 15]



$$S = 2^0 + 2^1 + 2^2 + \cdots + 2^{\log N}$$

$$= 2N$$

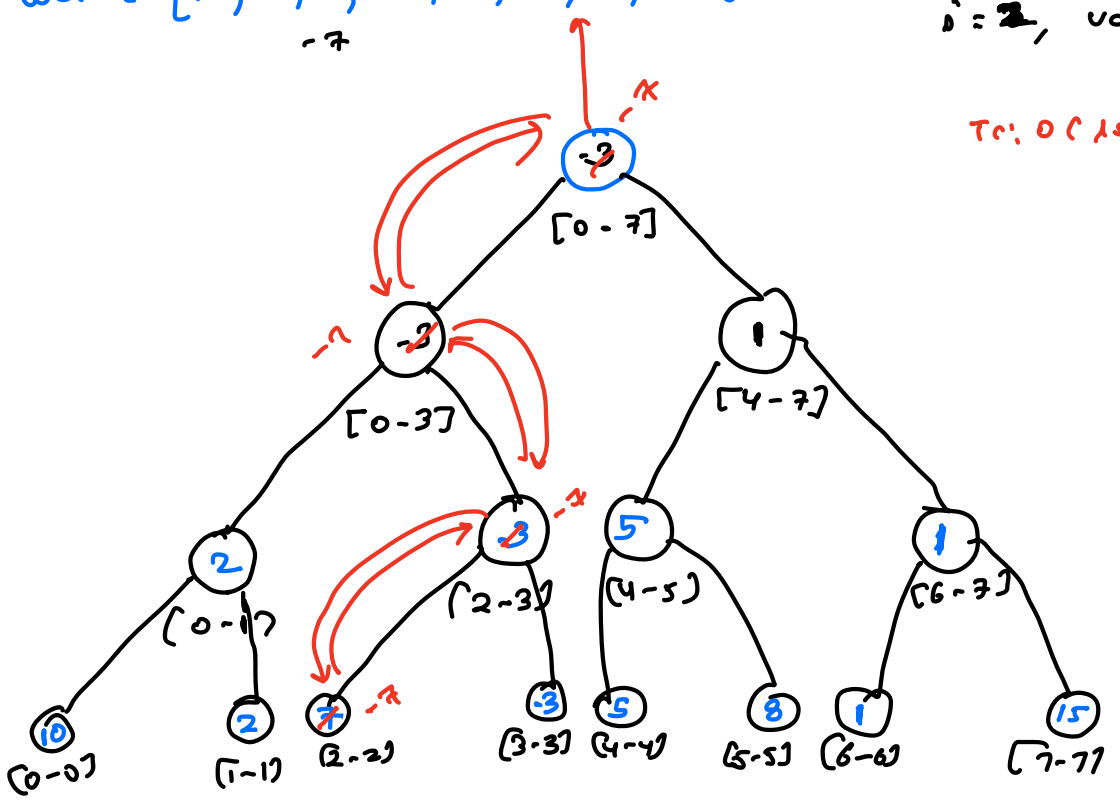
1. Build a tree
2. Update
3. Answer of query.

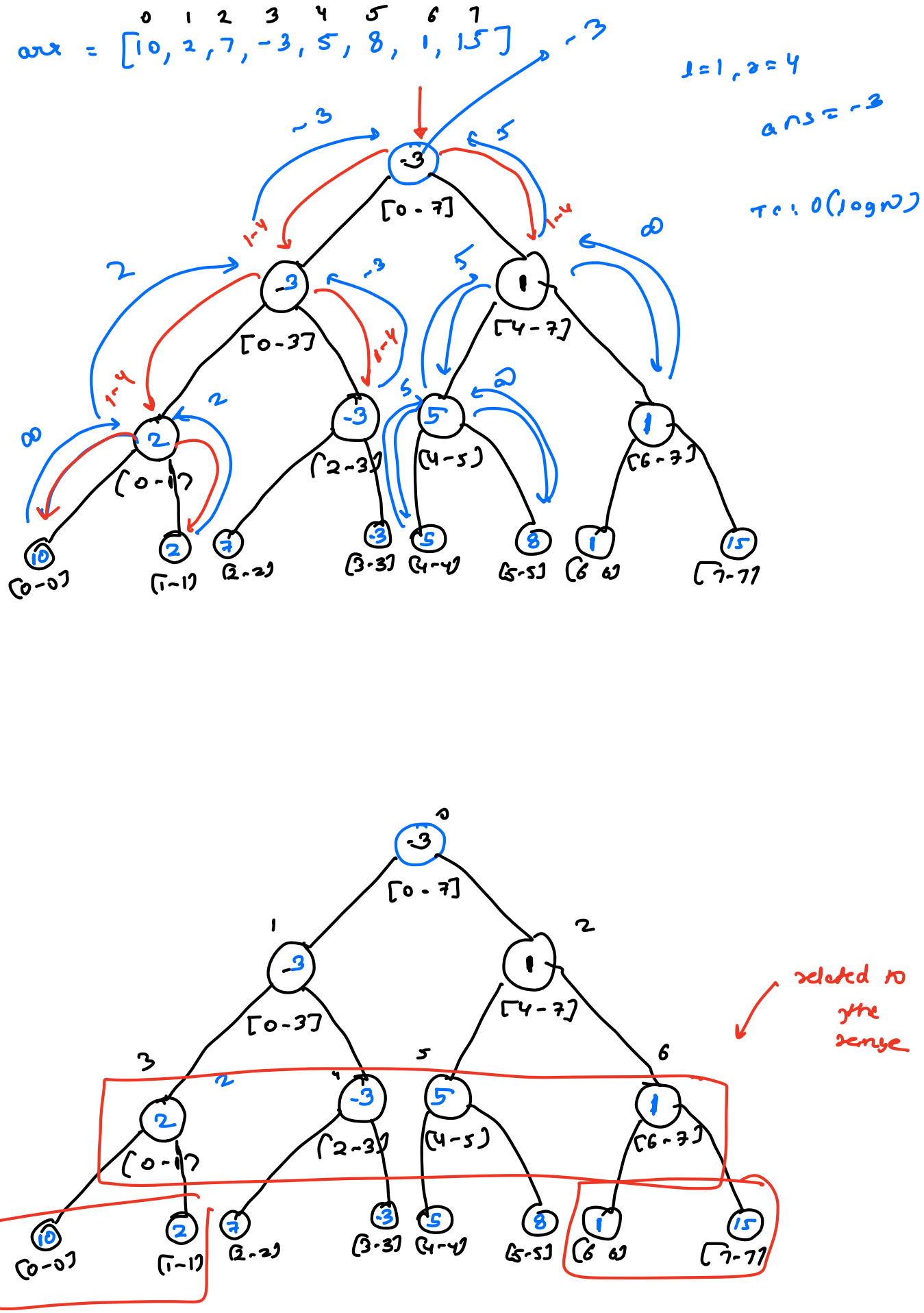


arr = [10, 2, 1, -3, 5, 8, 1, 15]
0 1 2 3 4 5 6 7

i = 2, val = -7

TC: O(logN)




0 1 2 3 4 5 6 7
arr = [10, 2, 7, -3, 5, 8, 1, 15]
l=1, r=4
ans = -3
TC: O(log N)
[0-7]
[0-3]
[4-7]
[0-1]
[2-3]
[4-5]
[6-7]
[0-0]
[1-1]
[2-2]
[3-3]
[4-4]
[5-5]
[6-6]
[7-7]
selected to the range

1. Build a tree
2. Update
3. Answer of query.

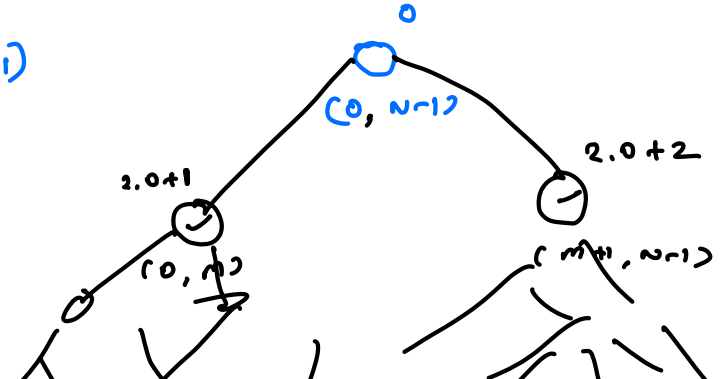
arr[N]

seg[4N] =

build(0, 0, N-1);

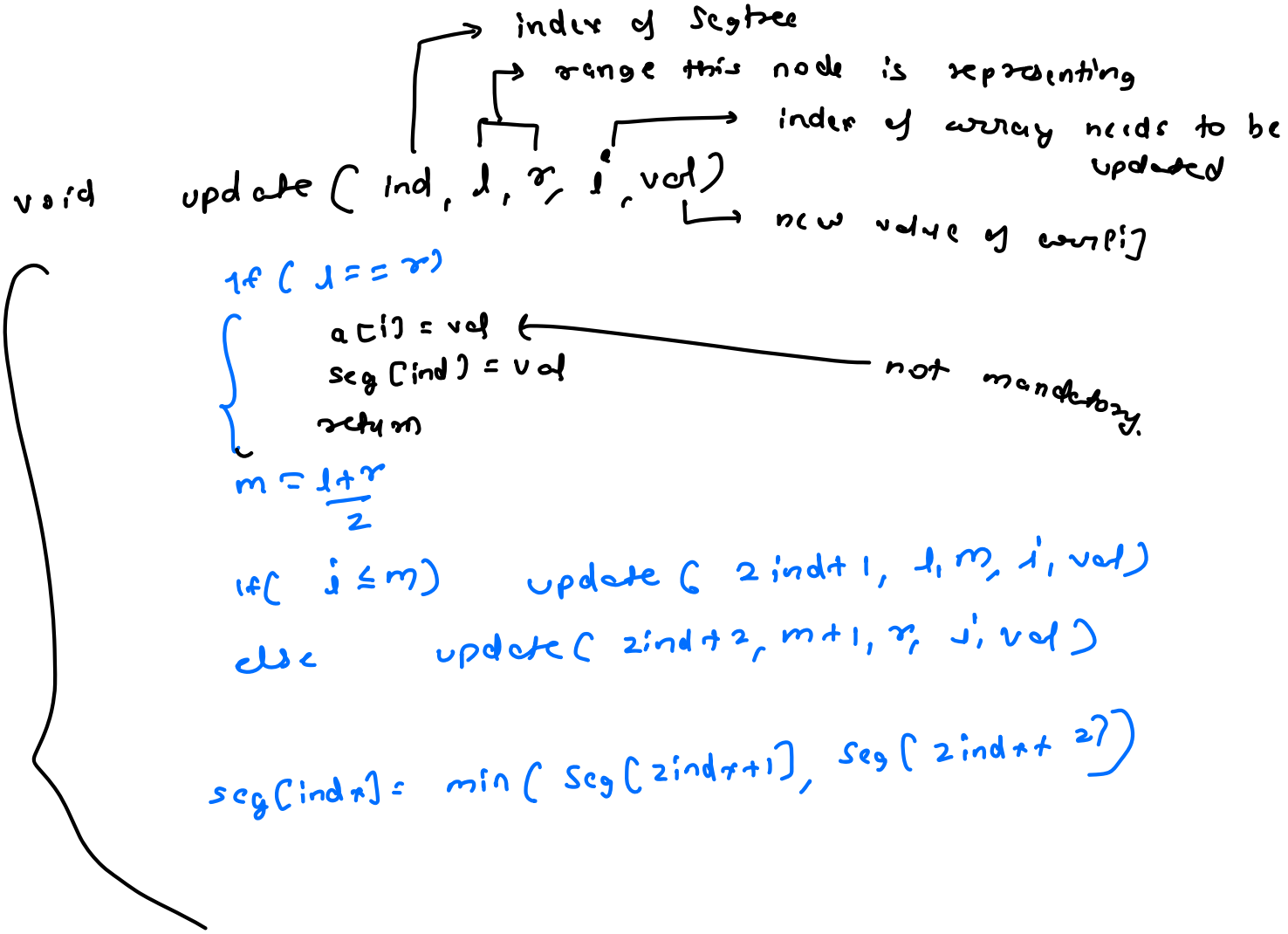
```
void build ( int indx, int l, int r)
{
    if( l == r)
    {
        seg[indx] = a[l]
        return
    }
    m = (l+r)/2
    build ( 2.indx+1, l, m)
    build ( 2.indx+2, m+1, r)
    seg[indx] = min ( seg[2indx+1], seg[2indx+2])
}
```

build(0, 0, N-1)



TC: $\theta(N)$

build(0, 0, N-1);



TC: O(log N)

index of segtree

range this node is representing

query(indx, l, r, ql, qr)

range query.

if(ql > r || qr < l) return INT_MAX

if(ql ≤ l && qr ≥ r) return seg[indx]

$m = \frac{l+r}{2}$

return min [query(2*indx+1, l, m, ql, qr)
query(2*indx+2, m+1, r, ql, qr)]

TC: O(log N)

TC: O(N + Q log N]

N → building the segtree

Q log N → Each query.

SC: O(4N)

↳ segtree[]

arr[N], Q [

build(0, 0, N-1)

for(each query)

{

if(Type 1)

{ update(0, 0, N-1, i, val)

}

else

{ printf(query(0, 0, N-1, l, r))

}

Q. Given an Array, Q queries

Each query.

1. i, val ⇒ update arr[i] = val

2. l, r ⇒ find gcd of index range [l, r]

```
void build ( int indx, int l, int r)
{
    if ( l == r)
    {
        seg[indx] = a[l]
        return
    }
    m = (l+r)/2
    build ( 2.indx+1, l, m)
    build ( 2.indx+2, m+1, r)
    seg[indx] = gcd ( seg[2indx+1], seg[2indx+2])
}
```

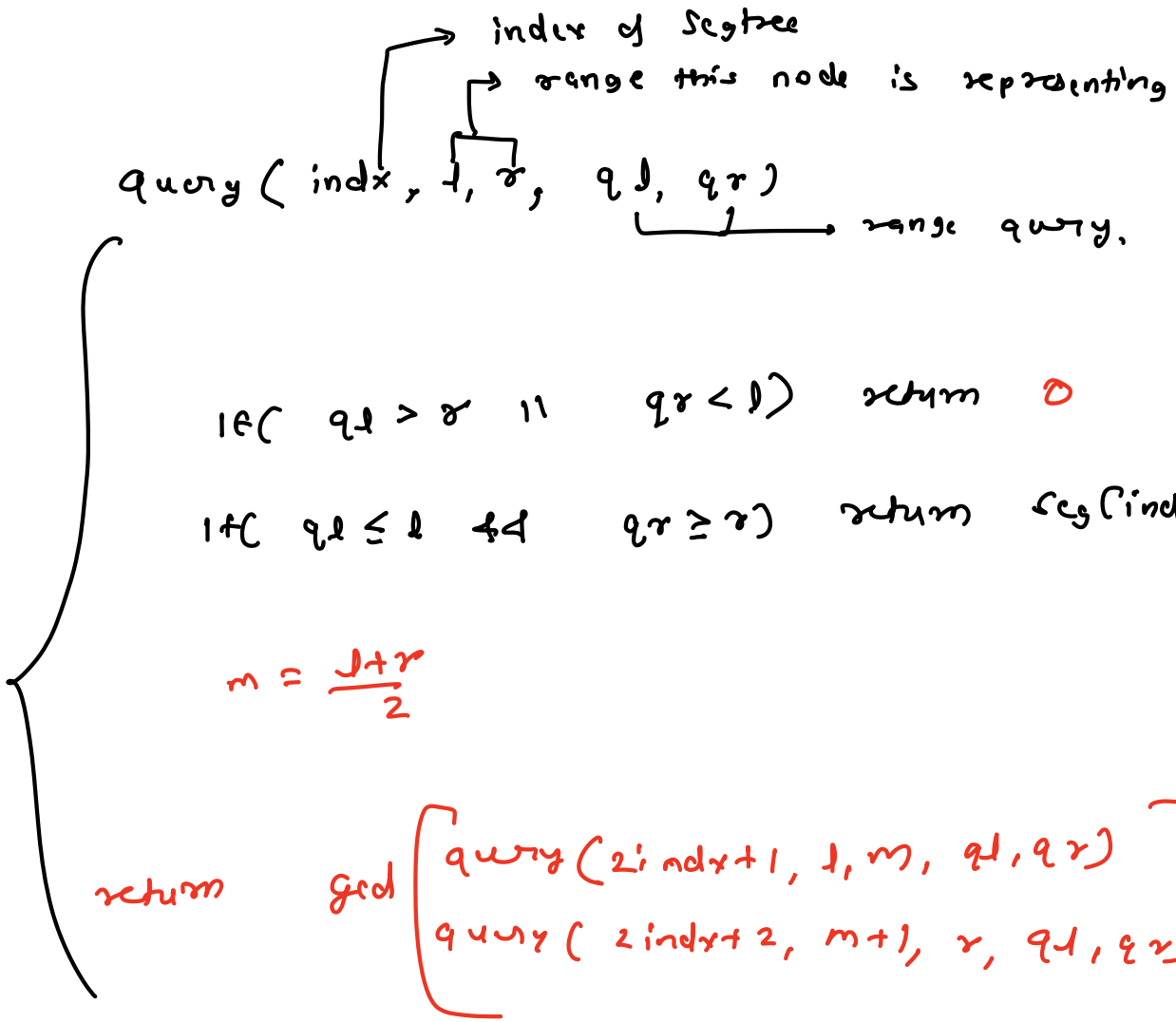
build(0, 0, N-1)

indx → index of segtree
l, r → range this node is representing
i → index of array needs to be updated
val → new value of arr[i]

```
void update ( ind, l, r, i, val)
{
    if ( l == r)
    {
        a[i] = val
        seg[ind] = val
        return
    }
    m = (l+r)/2
    if( i <= m)  update ( 2ind+1, l, m, i, val)
    else         update ( 2ind+2, m+1, r, i, val)

    seg[indx] = gcd ( seg[2indx+1], seg[2indx+2])
}
```

not mandatory.


index of segtree
range this node is representing
query (indx, l, r, ql, qr)
range query.
if(ql > r || qr < l) return 0
if(ql ≤ l && qr ≥ r) return seg[indx]
m = (l+r)/2
return gcd [query(2*indx+1, l, m, ql, qr)
query(2*indx+2, m+1, r, ql, qr)

gcd(0, x) = x